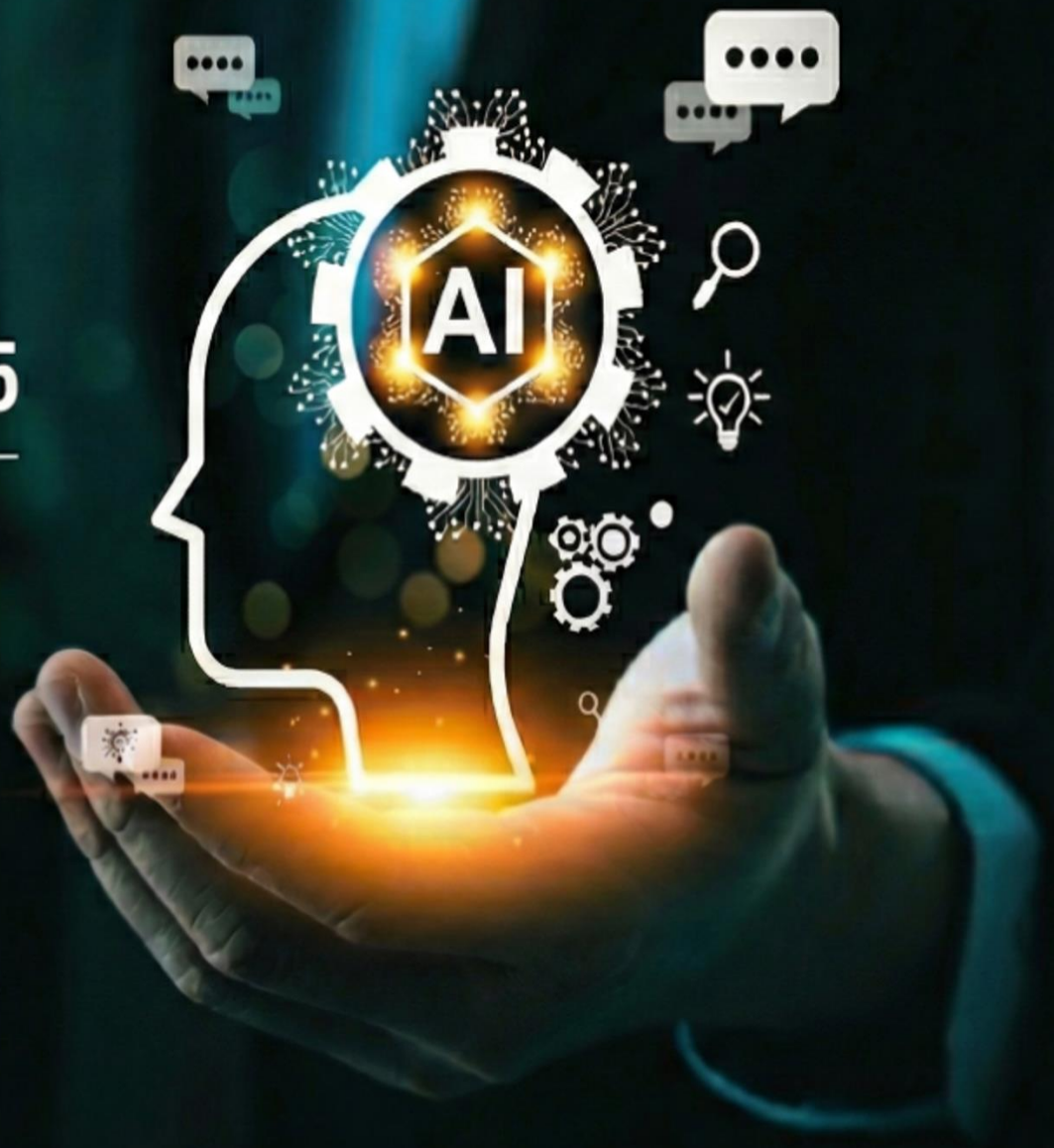
AGENTIC ARCHITECTURE DAY 2025

SEASON OF AI 5 MCP



Welcome to Agentic Architecture Day

✺ Part 1 - The Model Context Protocol (MCP) & Agentic Systems

🟣 14:30 - 15:00 — Check-in & Networking - Done

🎤 15:00 - 15:15 — Welcome & Introduction - Now

🌐 15:15 - 16:00 — The Model Context Protocol (MCP) in Action - Coming Soon

🟣 16:00 - 16:15 — Break

🏗️ 16:15 - 17:00 — Designing an MCP Server for Enterprise Systems

🟣 17:00 - 17:30 — Break & Transition to Part 2

What Is Season of AI

- Global community initiative
- Practical, hands-on learning around AI engineering
- Every season highlights a new capability
- Season 5 focus: Model Context Protocol (MCP)
- Activities include workshops, coding labs, and architecture sessions

Today's Theme: MCP

- MCP = Standard protocol for connecting AI models to tools, data, and systems
- Why MCP matters:
  - A unifying standard for agent–system interactions
  - Secure, controlled access to external capabilities
  - Enables enterprise-grade AI agents
  - Powers retrieval, actions, workflows, and automation
- Lets Begin…

Season of AI 5 MCP

Agentic Architecture Day

info@zion-net.co.il

Microsoft® Most Valuable Professional

Microsoft REGIONAL DIRECTOR

ZionNet

Agenda Overview

- Introduction
- The Model Context Protocol Philosophy
- The Players
- Using MCP Servers
- MCP Deep Dive
  - MCP Inspection Tools
  - MCP Packages

About Me

- Alon Fliess:
  - CTO of ZioNet
  - More than 40 years of hands-on experience
  - Microsoft Regional Director & Microsoft AI Foundry MVP
  - Azure Israel User Group Leader

About ZioNet

- A Professional Software service provider company
- Excellence in designing and implementing complex, high-performance software systems
- Deep expertise in cloud-native, distributed systems, IoT, .NET, C++, and cross-platform development
- Proven leadership in architecting and delivering enterprise-grade Agentic AI solutions
- Trusted by leading global companies for innovation, reliability, and engineering precision

Agent mode

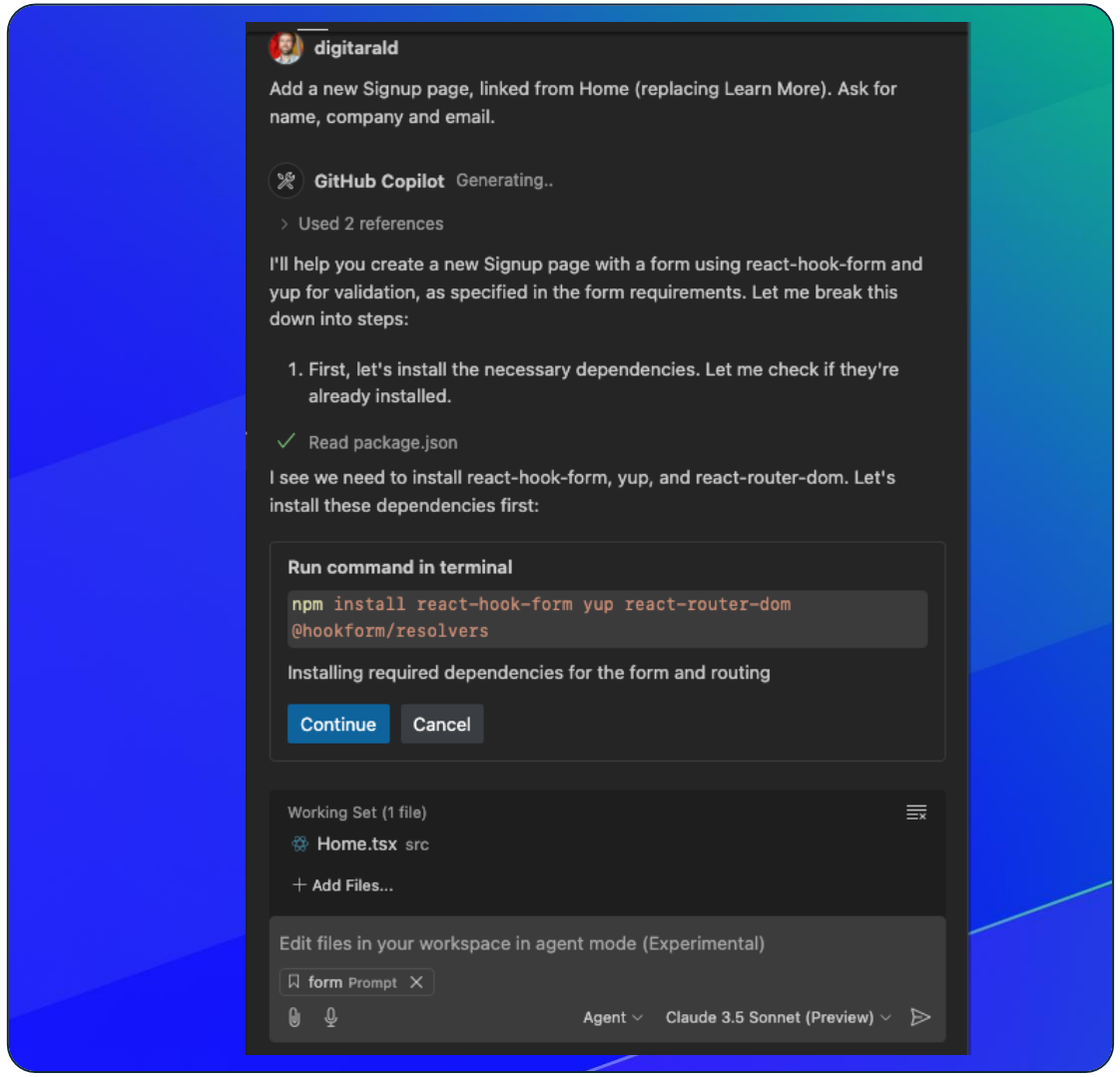
Agentic coding

As an Example of Agentic Systems

Accelerate delivery by shifting from code to outcomes

Autonomous AI pair programmer that **plans, executes, and iterates** on multi-step coding tasks – from editor to terminal, tests to debugger.

But how do we provide real time context?
How do we connect to our database? Query GitHub Issues? Run tests with natural language?
How can we bridge the gaps between our code, the client and LLM, and everything else?



The Model Context Protocol (MCP)

- The 'USB-C Port' for AI Applications
- What is MCP?
  - Open-source protocol (Anthropic, 2024)
  - Universal standard for AI ↔ tools/data
  - Adopted by: OpenAI, Google, Microsoft, and many others
  - Like Web API for AI
  - 🔗 What is the Model Context Protocol (MCP)? - Model Context Protocol
  - 🔗 https://github.com/microsoft/mcp-for-beginners

Model Context Protocol

Created by Anthropic
Adopted by the AI community
Evolving Together

ZioNet

The MCP Philosophy

- Universal Standardization ("USB-C for AI")
  - integration problem with a single open standard.
  - Decouples intelligence (Clients) from data (Servers), making them swappable.
- Radical Simplicity, Smart Host, Simple Server":
  - Pushes complexity to the application (Host).
  - Lowers the barrier to entry, enabling a massive ecosystem of lightweight tools.
- Fractal Composability
  - Client/Server distinction is logical, not physical.
  - Enables hierarchical agents where one system can act as both requester and provider.
- Contextual Isolation (Least Privilege) "Blind Servers":
  - Tools never see the full conversation history.
  - Servers operate only on specific data explicitly passed by the Host.
- Progressive Extensibility
  - Uses capability negotiation (Handshake) at startup.
  - Allows features like "Sampling" or "Notifications" to be added without breaking compatibility.

Parts of MCP

Hosts	Clients	Servers

Demo

- Configure and use MCP on:
  - VS Code
  - Visual Studio
  - Claude Desktop
  - ChatGPT

.vscode > {} mcp.json > ...

You, 2 days ago | 1 author (You)

```
1  {
2    "servers": {
       ✓Running | Stop | Restart | 11 tools | 7 prompts | More...
3      "windiag": {
4        "url": "http://localhost:5000/sse",
5        "headers": {
6          "X-API-Key": "secure-mcp-key"
7        }
8      }
9    }
10 }
```

The selected tools will be applied globally for all chat sessions that use the default agent.

- Built-In
  - changes Get diffs of changed files
  - edit Edit files in your workspace
  - extensions Search for VS Code extensions
  - fetch Fetch the main content from a web page. You should include the URL of the page you wa...
  - githubRepo Searches a GitHub repository for relevant source code snippets. You can specify a ...
  - new Scaffold a new workspace with VS Code-specific configurations to compile, debug and ru...
  - openSimpleBrowser Preview a locally hosted website in the Simple Browser
  - problems Check errors for a particular file
  - runCommands Runs commands in the terminal
  - runNotebooks Run notebook cells
  - runSubagent Runs a task within an isolated subagent context. Enables efficient organization o...
  - runTasks Runs tasks and gets their output for your workspace
  - runTests Runs unit tests (optionally with coverage)
  - search Search and read files in your workspace
  - testFailure Includes information about the last unit test failure
  - todos Tool for managing and tracking todo items for task planning
  - usages Find references, definitions, and other usages of a symbol
  - vscodeAPI Use VS Code API references to answer questions about VS Code extension develop...
- Azure MCP

MCP Deep-Dive

MCP Client

- Invokes **Tools**
- Queries for **Resources**
- Interpolates **Prompts**

MCP Server

- Exposes **Tools**
- Exposes **Resources**
- Exposes **Prompts**

Tools

Model-controlled

Functions invoked by the model

- Retrieve / search
- Send a message
- Update DB records

Resources

Application-controlled

Data exposed to the application

- Files
- Database records
- API Responses

Prompts

User-controlled

Pre-defined templates for AI interactions

- Document Q&A
- Transcript Summary
- Output as JSON

ZioNet

MCP Deep-Dive

MCP Server

- Invokes **Sampling** (LLM Use)
- Invokes **Elicitation**
- Receives **Roots**

Both Sides

- Invokes one-way **Notifications**
- Act upon **Notifications**

MCP Client

- Replies LLM result for **Sampling**
- Return **Elicitation** decisions
- Provides Access Rights with **Roots**

Sampling

Server-controlled

LLM completions requested by the Server

- Chain of Thought
- Self Corrections
- Multi-steps Plan

Elicitations

Server-controlled

Requiring user input or confirmation

- Confirm File Deletion
- Approve Email Send
- Buy a Product

Roots

Client-controlled

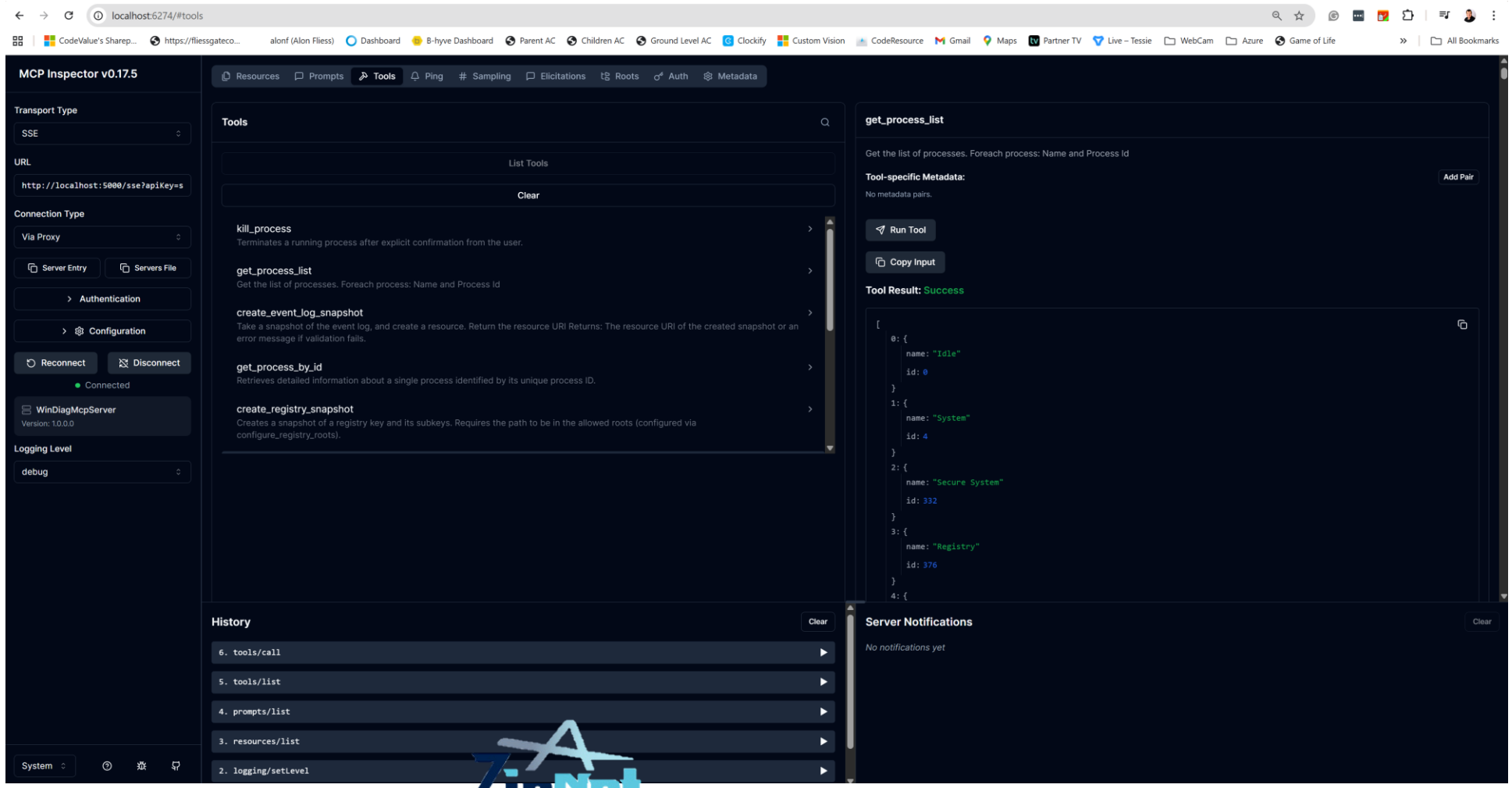
Boundaries for Server operations

- Restricts access to a volume
- Restricts sub-directory access
- Block Registry hive access

ZioNet

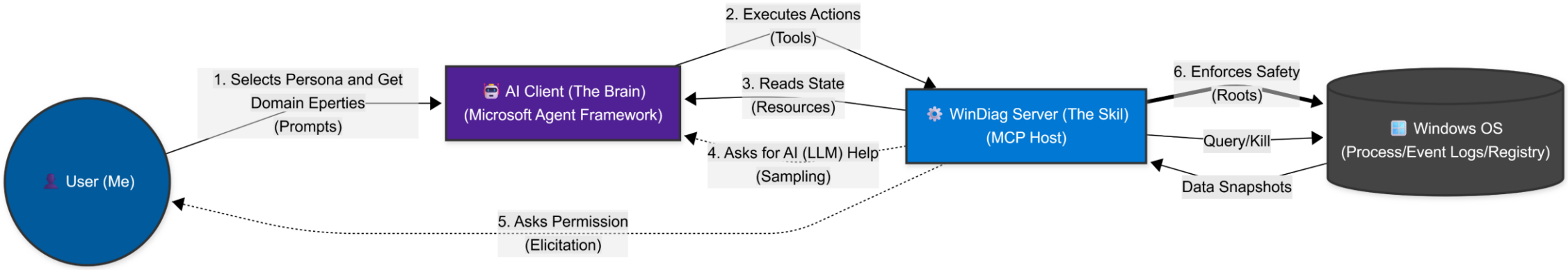
The MCP Inspector tool

- Like Postman for MCP



The Demo – The Big Picture

- The Demo is built from six milestones
- It is built with C#
  - However, all the concepts are part of the protocol and they are the same on all language's SDKs



Official C# SDK for MCP

Clients

Servers

Local – Stdio Transport

Remote – Server Sent Events

Remote – Streamable HTTP

NuGet Packages

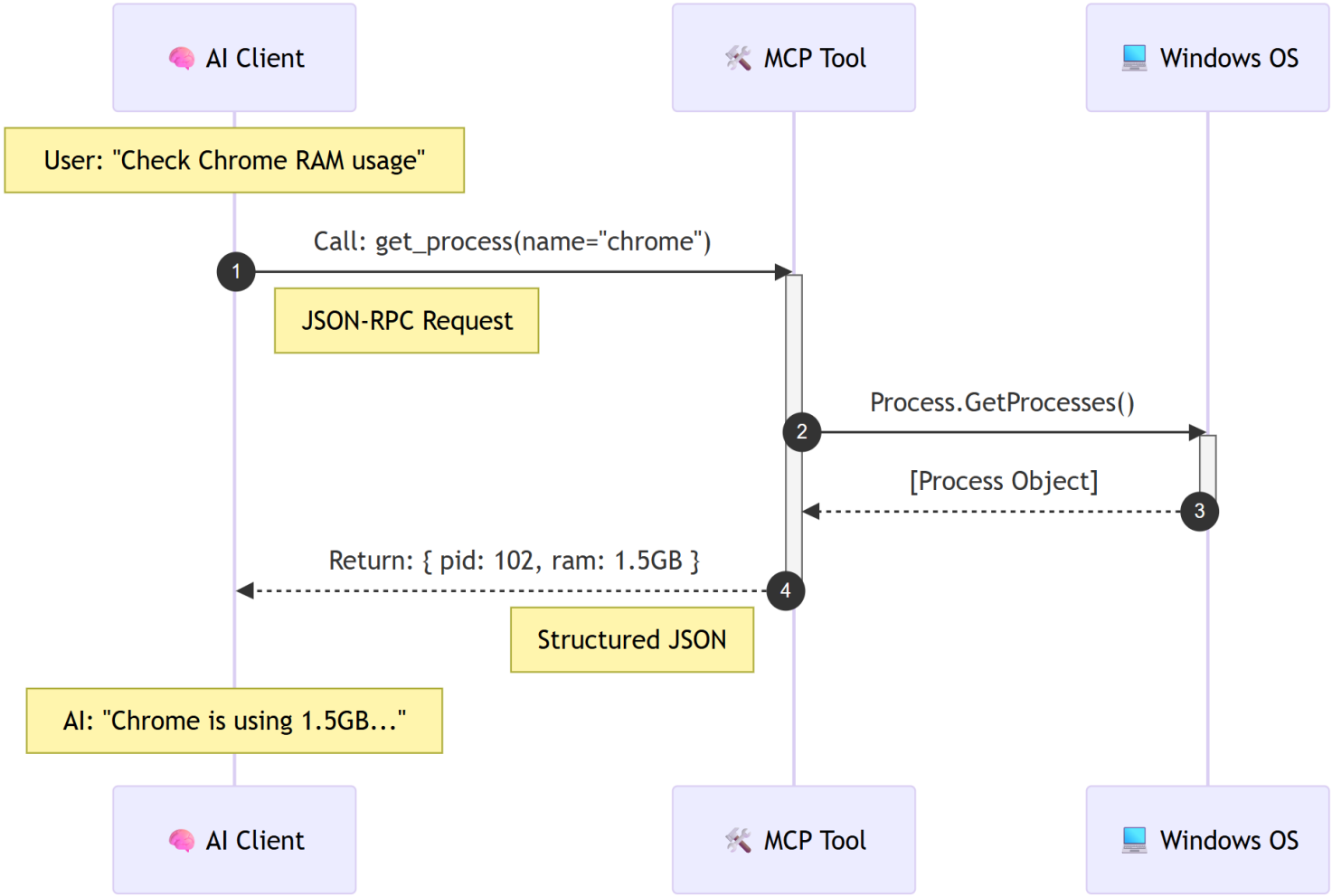
ModelContextProtocol.Core

ModelContextProtocol

ModelContextProtocol.AspNetCore

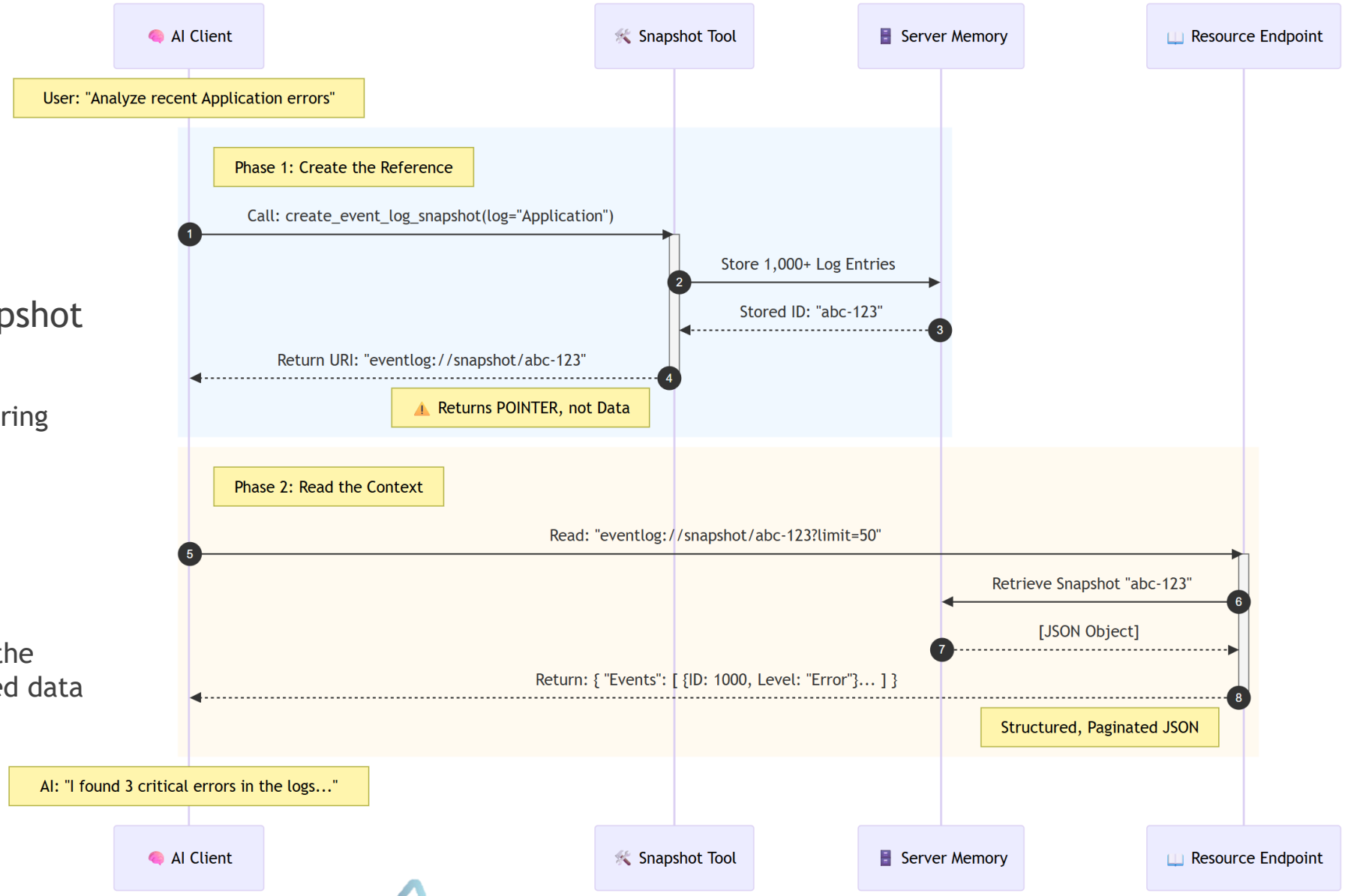
Tools Demo

- Milestone 1 – part 1:
  - A simple MCP Server
    - STDIO based protocol
    - The client builds and runs the server
    - A JSON-RPC message passing
    - The SDK takes care of all
- Milestone 2 – Part 2:
  - Add more process info tools



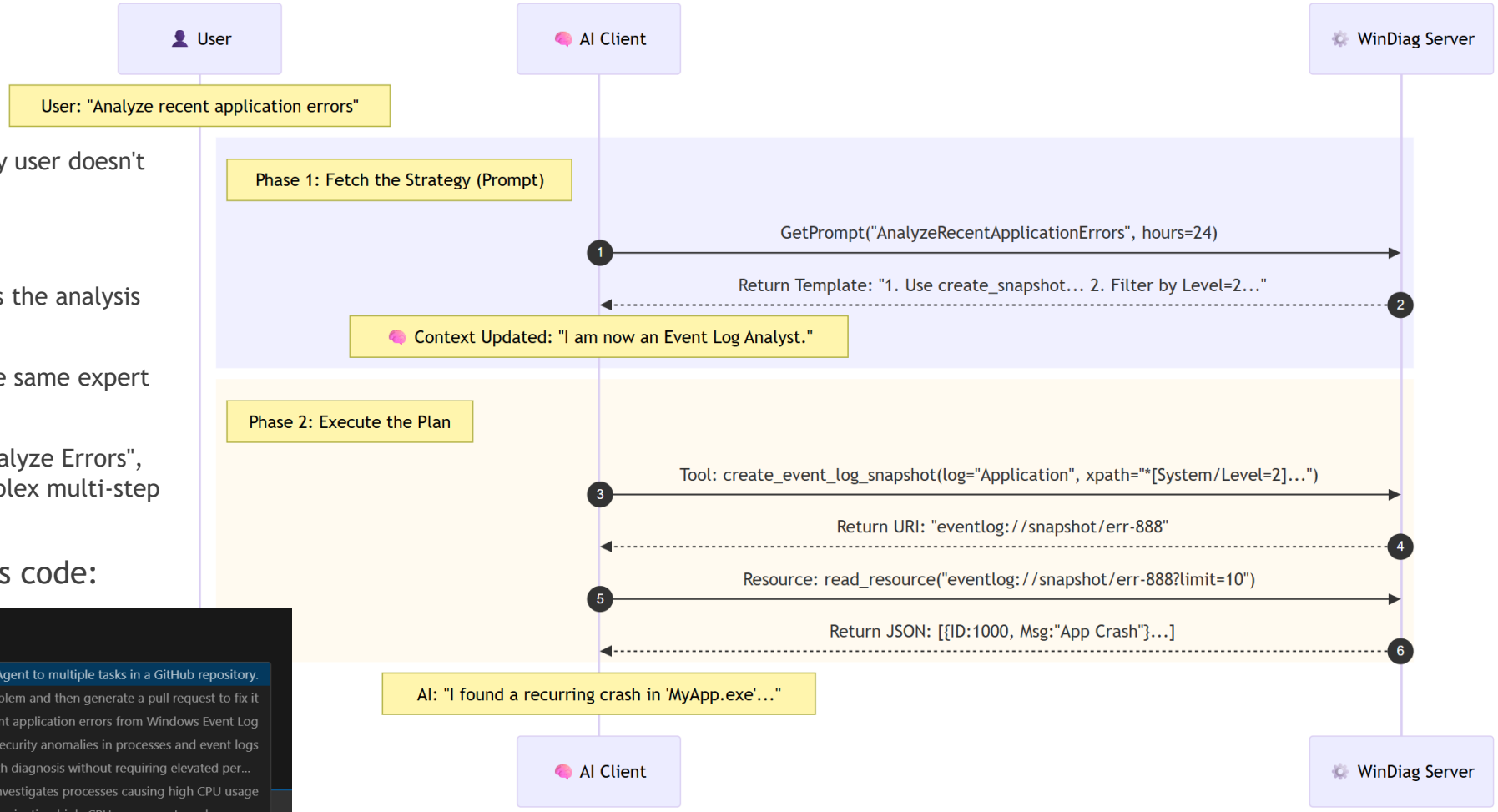
Resources Demo

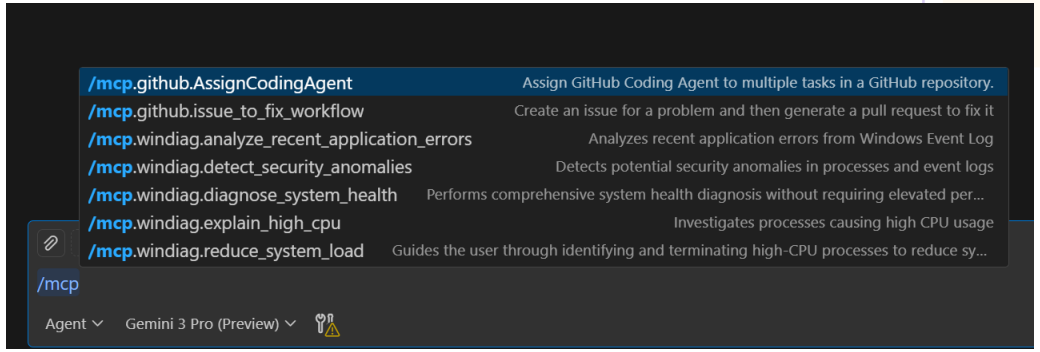
- Milestone 3 – Part 1:
  - Using Tool for a snapshot creation
    - Query string for filtering the massive data
  - Using Resource for snapshot content acquisition
    - Paging for reducing the amount of transferred data and used tokens



MCP Prompts – The Agent's Playbook - Demo

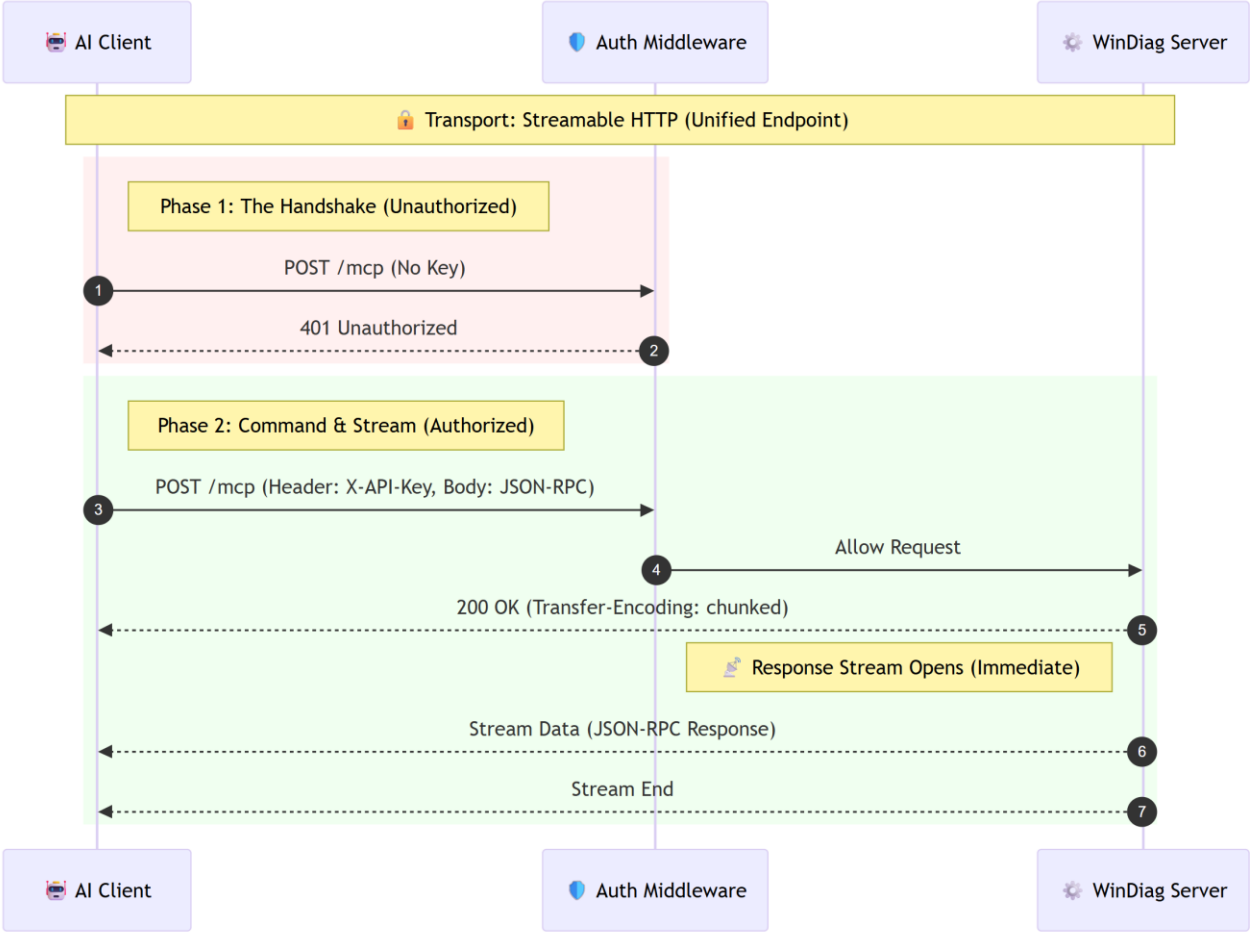
- Milestone 3 – Part 2:
  - The Gap:
    - "My user doesn't know XPath. My user doesn't know which tools to combine.“
  - The Solution:
    - Provide a prompt that describes the analysis process
    - Consistency: Every user gets the same expert analysis
    - Abstraction: The user types "Analyze Errors", but the system executes a complex multi-step technical workflow.
- “Run” Prompts with tools such as vs code:





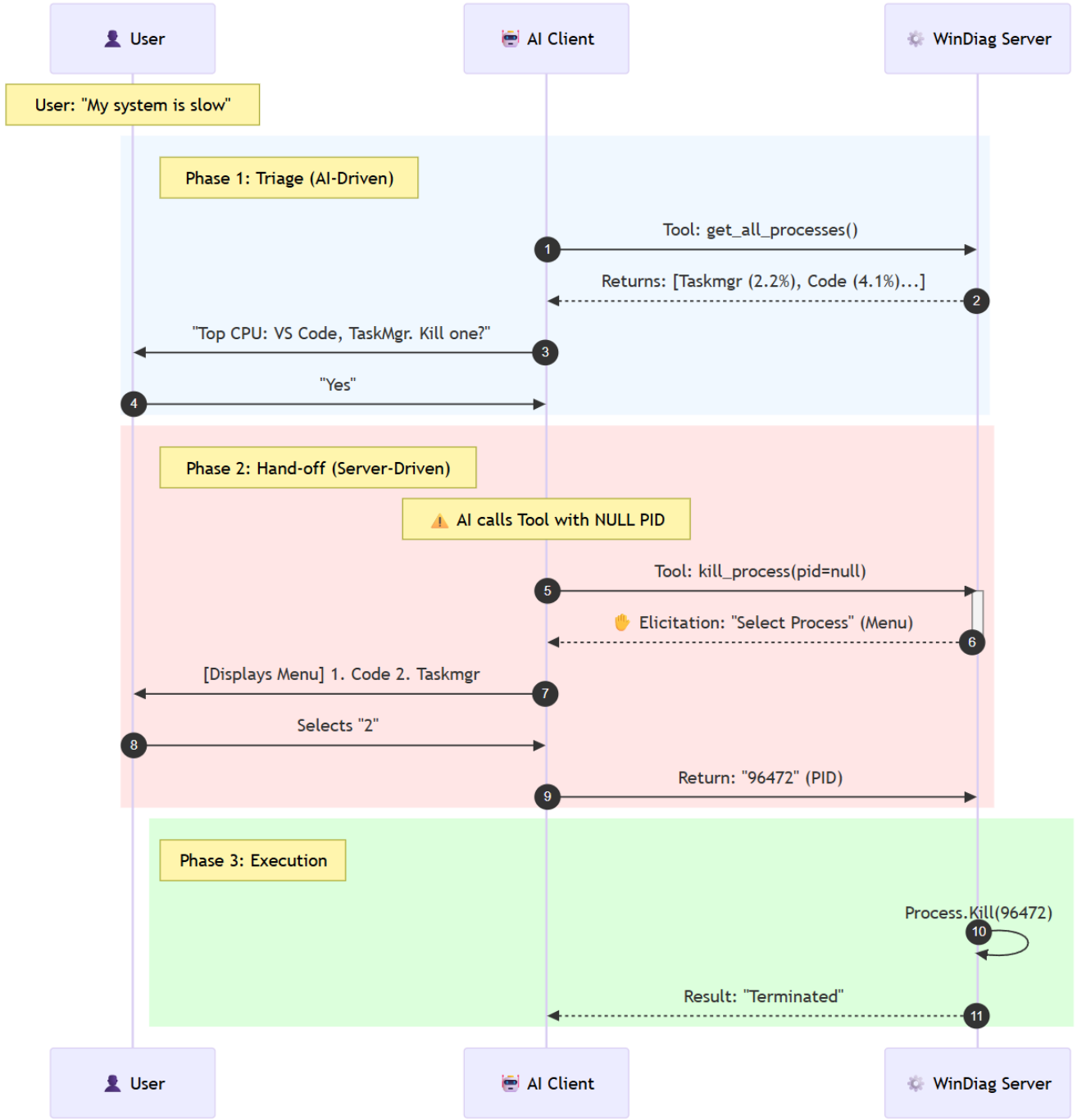
MCP Transport – Going Remote & Secure - Demo

- Milestone 4:
  - Using **Server-Side Event** – Legacy, considered deprecated
  - Using **Streamable HTTP** - New, efficient transform
- Comparison:
  - **Connections**: SSE = multiple/persistent; Streamable HTTP = single endpoint, efficient.
  - **Architecture**: SSE = asymmetric/complex; Streamable HTTP = unified/simpler.
  - **Scalability**: Streamable HTTP is more scalable and bette for cloud/Containers/VMs.
  - **Future**: Streamable HTTP is the future; SSE is deprecated.
- Authentication
  - API-Key based
  - OAuth-based



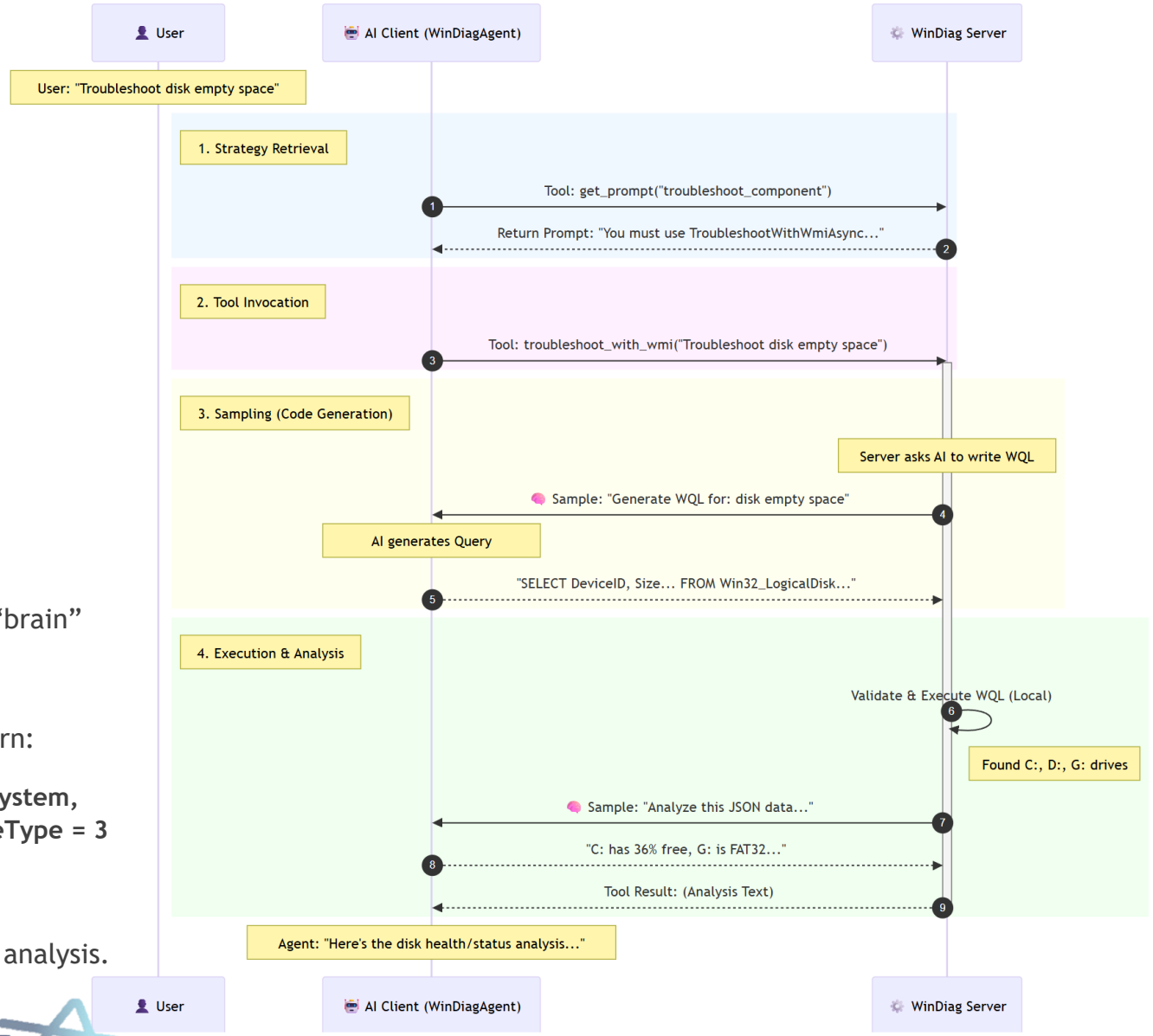
MCP Elicitation – The "Nuclear Launch Key"

- Milestone 5:
  - The user says: 'My system is slow'.
    - This is vague. It triggers a flow dictate by a prompt
    - The MCP Server uses Elicitation to communicate with the user



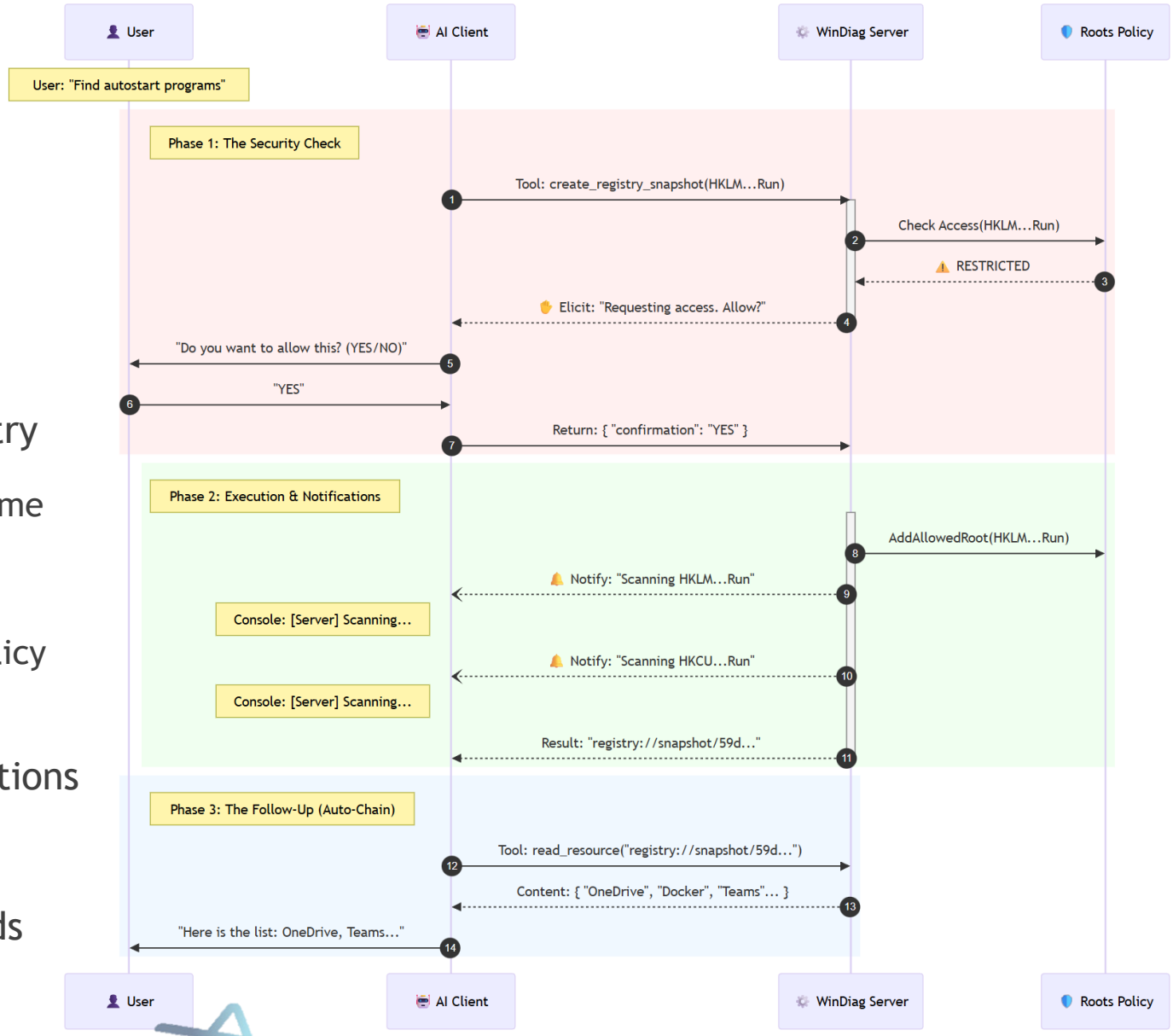
MCP Sampling – The Server's Brain Extension - Demo

- Milestone 6:
- The Server "Thinking“
  - User asks: "Troubleshoot disk empty space“
  - Agent loads the **troubleshoot_component** prompt.
    - The server follows the instructions.
  - The Inversion (**Sampling**):
    - The Server gets the request and prompt, but it needs a “brain”
    - It uses Sampling to ask the Client for a WMI query.
    - The client uses an agent to analyze the request and return:
      - **SELECT DeviceID, Size, FreeSpace, DriveType, FileSystem, VolumeName FROM Win32_LogicalDisk WHERE DriveType = 3**
  - The Server executes that query
    - It passes the raw bytes back to the AI using Sampling for analysis.
    - Final Output: A detail information about the disk usage



Security Boundaries & Live Feedback

- Milestone 7:
  - User: Find all autostart programs
  - The server needs to scan the registry
    - Resource Access: We use Just-in-Time User Consent (via Elicitation). In a server-side/headless scenario, this would be replaced by an Admin Policy or ACL.
  - The Server sends Scanning notifications
    - Progress updates
  - After the snapshot, the server reads the resulted resource



Demo Summary: The MCP Big-7 Structural Pillars

- Tools → Action
- Resources → Data
- Prompts → Templates
- Sampling → Agentic recursion
- Notifications → Live updates
- Roots → Sandboxing
- Elicitation → Human-in-the-loop

Model Context Protocol At Microsoft

Copilot Studio | VS Code | Foundry Agent | Visual Studio | Windows | GitHub Copilot | Microsoft Teams | Semantic Kernel

MCP Clients & Host

ATLASSIAN | neo4j | NEON

Tools

Community | Enterprise | Windows

Registry

C# MCP SDK | Azure API Management | Azure Functions | Microsoft Connectors

Build Local & Remote MCPs

ANTHROP\C | Agentic AI Foundation

MPC enables seamless Integration between LLM Apps and external data sources

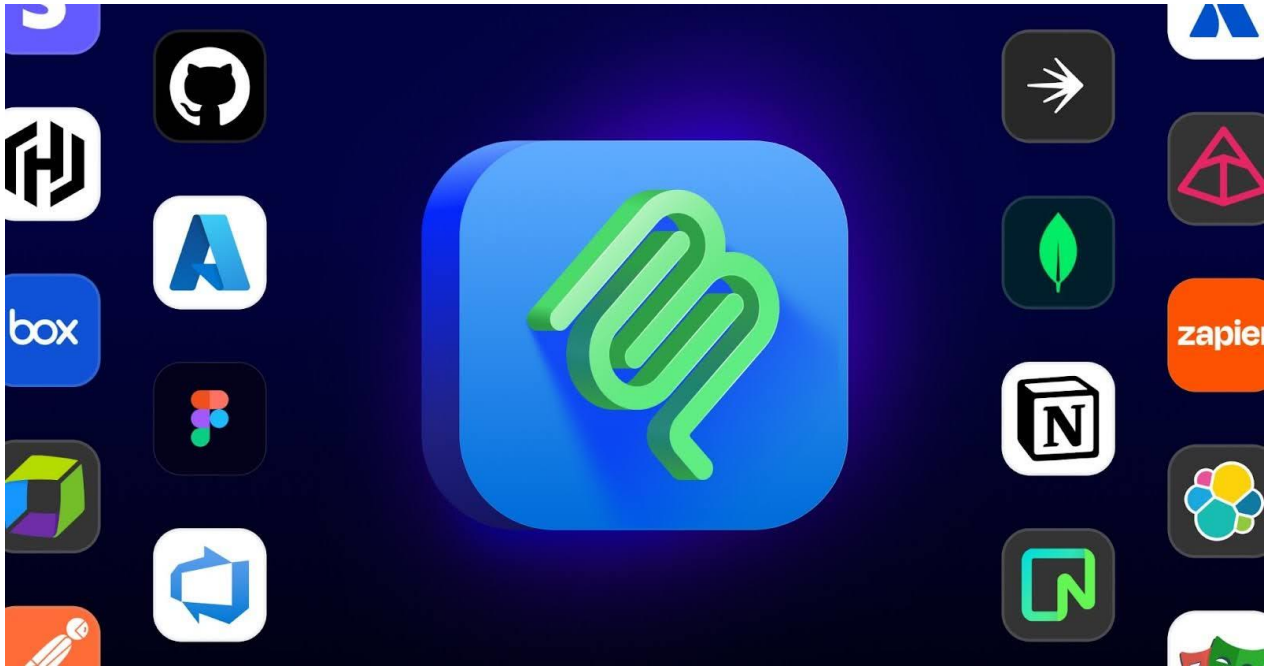
Model Context Protocol

ZioNet

GitHub MCP Registry

Servers and tools from the community that connect models to files, APIs, databases, and more.

- Launched September 2025
- Discoverable inside VS Code, one-click installs
- Works with any MCP-compatible host (including GitHub Copilot)
- Meant to reduce duplication and increase confidence when choosing and consuming MCP servers.



github.com/mcp

Learn more

Today's tutorial

https://github.com/alonf/MCPDemo

More training

aka.ms/mcp-for-beginners

Microsoft Learn MCP Server

aka.ms/Season-of-AI-MSLearnMCPServer

VS Code MCP Support

aka.ms/vscode-mcp

Model Context Protocol

modelcontextprotocol.io

Online AI Community

aka.ms/AIDiscord

Microsoft wants to hear from you!

Participate in the Season of AI post-event survey to share your feedback on this topic and content.

https://aka.ms/soai/mcp/attendeesurvey

Be an Architect